



Estructuras de control condicionales e iterativas en Python

Breve descripción:

El componente desarrolla las estructuras de control del lenguaje Python: condicionales (if, elif, else), operadores relacionales, lógicos, de inclusión y de asignación, e iterativas (while, for) junto con los operadores de rango para construir bucles.

Julio 2026

Tabla de contenido

Introducción	4
1. Estructuras condicionales: concepto, tipos, sintaxis, sentencias	7
1.1. Sintaxis general de las estructuras condicionales	8
1.2. Estructura if simple	11
1.3. Estructura if-else	14
1.4. Estructura if-elif-else	17
1.5. Estructuras condicionales anidadas	22
1.6. Operador ternario	24
1.7. Estructura match-case (Python 3.10+)	26
1.8. Tipos de estructuras condicionales	28
1.9. Buenas prácticas con estructuras condicionales	30
2. Operadores condicionales	32
2.1. Operadores relacionales	33
2.2. Operadores lógicos	37
2.3. Operadores de inclusión	41
2.4. Operadores de asignación	45
2.5. Operadores de identidad	49
2.6. Precedencia de los operadores condicionales	51

3. Estructuras iterativas: concepto, tipos, sintaxis, sentencias	57
3.1. Ciclo while	58
3.2. Ciclo for	62
3.3. Sentencias de control: break, continue y else	64
3.4. Ciclos anidados	68
4. Operadores de rango de datos: bucles	72
4.1. Función range().....	72
4.2. Uso de range() en ciclos for	75
4.3. Función enumerate()	78
4.4. Función zip()	81
4.5. Funciones reversed() y sorted()	84
Síntesis	87
Glosario	88
Referencias bibliográficas	89
Créditos	90

Introducción

Las estructuras de control son uno de los pilares de la programación. Permiten que un programa no se ejecute de forma lineal, sino que tome decisiones y repita tareas de acuerdo con los datos que recibe y con la lógica definida por el programador. Sin las estructuras de control sería imposible construir programas que respondan a situaciones variables, validen información o procesen grandes volúmenes de datos.

Python ofrece un conjunto sencillo y poderoso de estructuras de control. Las estructuras condicionales, encabezadas por las palabras reservadas `if`, `elif` y `else`, permiten al programa elegir entre distintos caminos según el resultado de una expresión booleana. Las estructuras iterativas, con las sentencias `while` y `for`, permiten ejecutar un bloque de instrucciones varias veces mientras se cumpla una condición o mientras existan elementos por recorrer en una secuencia.

Los operadores son los componentes que dan vida a las expresiones evaluadas por las estructuras de control. Los relacionales comparan valores; los lógicos combinan expresiones booleanas; los de inclusión verifican pertenencia en secuencias; y los de asignación permiten almacenar valores en variables. Su correcto uso garantiza que las decisiones del programa reflejen fielmente la lógica del problema a resolver.

Adicionalmente, los operadores y funciones de rango (principalmente `range()`, `enumerate()` y `zip()`) resultan esenciales para construir bucles controlados. Permiten generar secuencias de números, recorrer colecciones con sus índices y combinar varias secuencias en paralelo, lo que amplía considerablemente las posibilidades de los ciclos.

Para comprender la importancia del contenido y los temas abordados, se recomienda acceder al video introductorio:

Video 1. Estructuras de control condicionales e iterativas en Python



[Enlace de reproducción del video](#)

Transcripción del video. Estructuras de control condicionales e iterativas en Python

Programar en Python implica organizar instrucciones que permitan tomar decisiones, repetir procesos y responder a diferentes condiciones. Para lograrlo, se utilizan estructuras de control que orientan el camino del programa según los datos recibidos y la lógica definida para resolver un problema.

En este componente formativo se abordan las estructuras condicionales, su sintaxis general y sus principales formas de uso: if simple, if-else, if-elif-else, condiciones anidadas, operador ternario y estructura match-case. Estos elementos permiten representar rutas de decisión, validar información y ejecutar instrucciones de acuerdo con condiciones específicas.

Transcripción del video. Estructuras de control condicionales e iterativas en Python

El recorrido continúa con los operadores condicionales: relacionales, lógicos, de inclusión, de asignación y de identidad. También se explica la precedencia de los operadores, aspecto necesario para interpretar correctamente el orden en que Python evalúa las expresiones y evitar resultados inesperados.

Asimismo, se presentan las estructuras iterativas `while` y `for`, la comparación entre ambas, las sentencias de control `break`, `continue` y `else`, y los ciclos anidados. Estos recursos permiten repetir instrucciones de manera controlada y procesar datos en diferentes escenarios.

Finalmente, se estudian las funciones asociadas al trabajo con rangos y secuencias, como `range()`, `enumerate()`, `zip()`, `reversed()` y `sorted()`, junto con un ejemplo integrador de estructuras de control.

Así, las estructuras de control en Python fortalecen la capacidad para crear programas que tomen decisiones, automaticen tareas repetitivas y procesen datos de forma clara y ordenada, respondiendo a necesidades reales de desarrollo de software en contextos formativos, laborales, productivos y sociales del entorno colombiano.

1. Estructuras condicionales: concepto, tipos, sintaxis, sentencias

Las estructuras condicionales permiten que un programa tome decisiones durante su ejecución. Se utilizan cuando se necesita que ciertas instrucciones se ejecuten solo si se cumple una condición específica. De esta manera, el flujo del programa deja de ser lineal y se adapta a diferentes situaciones según los datos que recibe. Sin las estructuras condicionales, un programa solo podría ejecutar una secuencia fija de instrucciones, lo que limitaría enormemente su utilidad práctica.

Antes de ver ejemplos aplicados, conviene fijar cuatro ideas clave que sostienen el funcionamiento de toda estructura condicional en Python:

- **Palabras reservadas:** `if`, `elif` y `else` construyen las estructuras condicionales en Python, cada una con un rol específico dentro de la decisión.
- **Expresión booleana:** la condición de toda estructura condicional se expresa así y, al evaluarse, produce siempre un valor `True` o `False`.
- **Evaluación:** si el resultado es `True` se ejecuta el bloque asociado; si es `False`, se omite o se evalúa la siguiente condición.
- **Indentación:** a diferencia de C o Java, Python no usa llaves; delimita los bloques con espacios, exigiendo una estructura visual clara.

Estos cuatro elementos muestran que toda estructura condicional en Python depende de una sintaxis precisa: una palabra reservada, una condición booleana, una regla de evaluación y una indentación correcta que trabajan en conjunto para que el programa tome decisiones de forma confiable.

A partir de esta comprensión, resulta necesario reconocer la sintaxis con la que cada estructura condicional se construye en Python. En el siguiente apartado, se

presenta la estructura if simple, la más sencilla de todas, la cual permite ejecutar un bloque de instrucciones únicamente cuando se cumple una condición.

1.1. Sintaxis general de las estructuras condicionales

La sintaxis básica de una estructura condicional en Python utiliza la palabra reservada **if** seguida de la condición y dos puntos (:). Las instrucciones que se ejecutan cuando la condición es verdadera deben estar indentadas con cuatro espacios. Python utiliza la indentación para definir el bloque de código que pertenece a la estructura, por lo que una indentación incorrecta genera un error de tipo `IndentationError`.

La indentación, lejos de ser un detalle de estilo, es el mecanismo con el que Python reconoce qué instrucciones forman parte de una decisión. En el trabajo profesional, mantener una sangría coherente facilita que otras personas lean y modifiquen el código, y previene errores difíciles de detectar cuando los bloques se vuelven extensos o se combinan entre sí.

Dominar esta forma básica es el punto de partida para construir cualquier decisión dentro de un programa, pues sobre ella se apoyan todas las variantes condicionales del lenguaje.

Por ello, conviene observar con atención tres elementos:

- **Palabra reservada if:** inicia toda estructura condicional simple e indica a Python que debe evaluar una condición antes de continuar.
- **Condición evaluada:** expresión booleana ubicada entre **if** y los dos puntos, que determina si el bloque siguiente se ejecuta.

- **Bloque indentado:** conjunto de instrucciones con cuatro espacios de sangría que se ejecutan únicamente cuando la condición evaluada resulta verdadera.

Estos tres elementos, la palabra reservada, la condición y el bloque indentado, conforman la estructura mínima que cualquier programa en Python necesita para tomar una decisión.

Con la estructura clara, es momento de observar cómo estos tres elementos se combinan en la sintaxis real de Python: primero en su forma general, luego en un ejemplo aplicado y, finalmente, en un caso donde la condición no requiere una comparación explícita:

- **Código. Sintaxis general de la estructura condicional if**

if condicion:

 instrucción_1

 instrucción_2

...

 instrucción_n

Cuando la condición se evalúa como True, Python ejecuta las instrucciones indentadas. Cuando es False, las instrucciones se omiten y el flujo continúa con la siguiente línea sin indentación. Es importante notar que los dos puntos al final de la línea del if son obligatorios, olvidarlos produce un SyntaxError.

- **Código. Estructura condicional if**

```
>>> edad = 20

>>> if edad >= 18:

...     print('Es mayor de edad')

...

Es mayor de edad
```

En Python, una condición no necesariamente tiene que ser una comparación explícita; cualquier expresión que produzca un valor puede utilizarse como condición. En ese caso, Python aplica sus reglas de conversión a booleano: valores como 0, 0.0, cadenas vacías "", listas vacías [], None y False se consideran falsos; todo lo demás se considera verdadero.

- **Código. Condiciones sin comparación explícita**

```
# Ejemplos de condiciones sin comparación explícita

>>> nombre = 'Ana'

>>> if nombre:

...     print('El nombre no está vacío')

...

El nombre no está vacío

>>> lista = []

>>> if not lista:
```

```
... print('La lista está vacía')
```

```
...
```

La lista está vacía

La cadena 'Ana' se evalúa como verdadera por no estar vacía, mientras que la lista vacía se evalúa como falsa, sin necesidad de comparar su longitud con cero.

Estos tres ejemplos evidencian que la sintaxis if condición, se aplica de la misma manera sin importar si la condición proviene de una comparación explícita o de la evaluación directa de un valor, siempre delimitando el bloque con indentación.

La sintaxis anterior constituye la base para aplicar la forma más sencilla de decisión. A continuación, se estudia la estructura if simple y su ejecución cuando una condición es verdadera.

1.2. Estructura if simple

La estructura if simple evalúa una única condición. Si la condición es verdadera, ejecuta el bloque de instrucciones asociado; si es falsa, el programa continúa sin ejecutar nada adicional. Es la estructura condicional más elemental y se utiliza cuando solo interesa actuar ante un caso específico, sin necesidad de hacer nada cuando la condición no se cumple.

Esta clase de decisión aparece de forma constante en el desarrollo de software: validar que un campo no esté vacío, conceder un permiso solo si se cumple un requisito o mostrar una alerta cuando un valor supera cierto límite son situaciones cotidianas

que se resuelven con un if simple. Su sencillez lo convierte en la primera herramienta de control que utiliza cualquier programador. Para emplearla con seguridad, conviene distinguir dos elementos:

- **Condición:** expresión que siempre produce un valor verdadero o falso, determinando si el bloque debe ejecutarse.
- **Bloque indentado:** conjunto de instrucciones que se ejecuta únicamente cuando la condición se cumple.

Con la condición y el bloque indentado claros, el if simple queda listo para aplicarse en casos reales de validación.

A continuación, se presentan dos ejemplos que aplican la estructura if simple en situaciones frecuentes de la práctica profesional: el control de acceso por edad y la aplicación de descuentos por compra:

Ejemplo 1. Aplicar un if simple para validar si una persona cumple con la edad mínima requerida antes de permitirle el acceso a un sistema.

- **Código. Programa: validar edad mínima**

```
# Programa: validar edad mínima

edad = int(input('Ingrese su edad: '))

if edad >= 18:

    print('Puede ingresar al sistema')

print('Gracias por su visita')
```

En este ejemplo, el mensaje 'Puede ingresar al sistema' se imprime únicamente cuando la condición $\text{edad} \geq 18$ es verdadera, pues se encuentra dentro del bloque indentado. En cambio, el mensaje 'Gracias por su visita' se imprime siempre, ya que está fuera de dicho bloque y no depende de la condición. Este comportamiento ilustra cómo la indentación delimita con precisión el alcance de la estructura condicional.

Ejemplo 2. Ilustrar el uso del if simple en un contexto de control de descuentos.

- **Código. Programa: aplicar descuento por compra**

```
# Programa: aplicar descuento por compra mayor a $100.000

total_compra = float(input('Total de la compra: '))

descuento = 0

if total_compra > 100000:

    descuento = total_compra * 0.10

    print(f'Aplica descuento del 10%: ${descuento:,.0f}')

total_final = total_compra - descuento

print(f'Total a pagar: ${total_final:,.0f}')
```

En este caso, el programa calcula un descuento del 10 % solamente cuando la compra supera los 100 mil pesos. Si no se cumple la condición, el descuento permanece en cero y el total final es igual al total de la compra original. Este patrón es muy frecuente en sistemas de facturación.

Con el manejo del if simple ya dominado, es momento de conocer la estructura if-else, que permite definir una segunda ruta de acción cuando la condición no se cumple.

1.3. Estructura if-else

La estructura if-else permite ejecutar un bloque de instrucciones cuando la condición es verdadera y otro bloque diferente cuando es falsa. De esta manera, el programa siempre ejecuta uno de los dos caminos posibles, garantizando una respuesta en cualquier situación. Se utiliza cuando las dos alternativas son mutuamente excluyentes.

La estructura if-else resulta especialmente útil en decisiones que solo admiten dos caminos posibles. Algunos ejemplos frecuentes en el desarrollo de software son:

- **Aprobación o rechazo:** el programa acepta o rechaza una solicitud según se cumpla o no un requisito.
- **Permiso o denegación:** un sistema permite o deniega el acceso a un recurso según las credenciales ingresadas.
- **Descuento o precio completo:** una tienda aplica una rebaja o cobra el valor íntegro según el monto de la compra.

En los tres casos, el programa evalúa una sola condición y responde siempre con una de dos acciones posibles, sin dejar la decisión sin resolver.

A continuación, se presentan tres ejemplos que aplican la estructura if-else en situaciones distintas: clasificar un número como par o impar, verificar el derecho al voto y calcular el valor absoluto de un número:

Utilizar el operador de módulo para determinar si un número ingresado es par o impar.

- **Código. Sintaxis de la estructura if-else**

if condicion:

 # bloque cuando es verdadera

 instrucciones_si_verdadero

else:

 # bloque cuando es falsa

 instrucciones_si_falso

Programa: determinar si un número es par o impar

numero = int(input('Ingrese un número: '))

if numero % 2 == 0:

 print(f'{numero} es un número par')

else:

 print(f'{numero} es un número impar')

El operador % (módulo) devuelve el residuo de la división. Si el residuo al dividir entre 2 es cero, el número es par; de lo contrario, es impar. La estructura if-else garantiza que siempre se imprima uno de los dos mensajes.

Presentar cómo determinar si una persona puede votar según su edad y su nacionalidad.

- **Código. Programa: verificar derecho al voto**

```
# Programa: verificar derecho al voto

edad = int(input('Edad: '))

nacionalidad = input('Nacionalidad: ').lower()

if edad >= 18 and nacionalidad == 'colombiana':

    print('Tiene derecho al voto en Colombia')

else:

    print('No cumple los requisitos para votar en Colombia')
```

Explicar el cálculo del valor absoluto de un número, utilizando solamente las estructuras condicionales básicas (sin usar la función abs()).

- **Código. Programa: calcular el valor absoluto**

```
# Programa: calcular el valor absoluto de un número

numero = float(input('Ingrese un número: '))

if numero < 0:
```



```
valor_absoluto = -numero

else:

    valor_absoluto = numero

print(f'El valor absoluto es: {valor_absoluto}')
```

La estructura if-else garantiza siempre una respuesta del programa, sin dejar casos sin resolver, sin importar qué tan compleja sea la condición evaluada.

La estructura if-else resuelve decisiones con dos caminos. Para situaciones que requieren evaluar tres o más alternativas, se emplea la estructura if-elif-else.

1.4. Estructura if-elif-else

Cuando un programa debe evaluar varias condiciones posibles, se utiliza la estructura if-elif-else. La palabra reservada elif (abreviatura de else if) permite encadenar múltiples condiciones que se evalúan en orden. Python ejecuta el bloque asociado a la primera condición que resulte verdadera y omite el resto. Si ninguna condición se cumple, se ejecuta el bloque else, que es opcional.

Muchos problemas no se reducen a dos opciones, sino que exigen elegir entre varias categorías:

- Clasificar una nota.
- Asignar un rango de edad.
- Determinar una tarifa según el horario.

En estos casos, la estructura if-elif-else permite expresar todas las alternativas de forma ordenada, evitando una maraña de condiciones sueltas.

Para aprovecharla correctamente conviene recordar que las condiciones se evalúan en orden y que solo se ejecuta el primer bloque verdadero. Por ello, el orden en que se escriben las condiciones influye en el resultado, aspecto que los ejemplos de esta sección permiten apreciar con claridad:

A. Sintaxis general de la estructura if-elif-else y su aplicación para clasificar una calificación según varios rangos.

Código. Sintaxis de la estructura if-elif-else

```
if condicion_1:
    bloque_1
elif condicion_2:
    bloque_2
elif condicion_3:
    bloque_3
else:
    bloque_por_defecto

# Programa: clasificar una calificación

nota = float(input('Ingrese la nota (0 a 5): '))
```

```
if nota >= 4.5:

    print('Excelente')

elif nota >= 4.0:

    print('Sobresaliente')

elif nota >= 3.0:

    print('Aprobado')

else:

    print('Reprobado')
```

El código compara la nota con varios rangos. Si es igual o superior a 4,5, muestra “Excelente”; desde 4,0, “Sobresaliente”; desde 3,0, “Aprobado”; y, si es inferior a 3,0, “Reprobado”.

B. Clasificación del índice de masa corporal (IMC), una aplicación clásica en el área de salud.

Código. Programa: clasificar el IMC

```
# Programa: clasificar el IMC de una persona

peso = float(input('Peso en kilogramos: '))

estatura = float(input('Estatura en metros: '))

imc = peso / (estatura ** 2)

if imc < 18.5:

    categoria = 'Bajo peso'
```

```
elif imc < 25.0:

    categoria = 'Peso saludable'

elif imc < 30.0:

    categoria = 'Sobrepeso'

elif imc < 35.0:

    categoria = 'Obesidad grado I'

elif imc < 40.0:

    categoria = 'Obesidad grado II'

else:

    categoria = 'Obesidad grado III'

print(f'IMC: {imc:.2f} - Categoría: {categoria}')
```

El código calcula el IMC con el peso y la estatura. Luego, clasifica el resultado desde «Bajo peso» hasta «Obesidad grado III», según el rango obtenido.

C. Cálculo de la tarifa de un taxi según el horario del servicio.

```
Código. Programa: calcular tarifa de taxi

# Programa: calcular tarifa de taxi según horario

hora = int(input('Hora del servicio (0-23): '))

distancia = float(input('Distancia en km: '))
```

```
if 6 <= hora < 20:

    tarifa_base = 5000

    valor_km = 1500

    tipo = 'Tarifa diurna'

elif 20 <= hora < 23:

    tarifa_base = 6000

    valor_km = 2000

    tipo = 'Tarifa nocturna'

else:

    tarifa_base = 8000

    valor_km = 2500

    tipo = 'Tarifa de madrugada'

total = tarifa_base + (valor_km * distancia)

print(f'{tipo}: ${total:,.0f}')
```

El código selecciona la tarifa diurna, nocturna o de madrugada según la hora del servicio. Luego, calcula el valor total con la tarifa base, el precio por kilómetro y la distancia recorrida.

Los ejemplos muestran que la estructura if-elif-else permite organizar varias alternativas y ejecutar únicamente la primera condición verdadera. El orden de evaluación resulta esencial para clasificar los datos de manera correcta.

1.5. Estructuras condicionales anidadas

Una estructura condicional puede contener otras estructuras condicionales en su interior. A esto se le conoce como anidamiento. Es útil cuando la decisión a tomar depende de varias condiciones relacionadas que se evalúan en niveles. Al anidar, es muy importante respetar los niveles de indentación para que Python identifique correctamente a qué bloque pertenece cada instrucción.

Para interpretarlas con claridad conviene leerlas por niveles: se evalúa la condición exterior y, solo si se cumple, se valoran las condiciones internas que dependen de ella. Este razonamiento por etapas es el que permite seguir la lógica de los programas de validación.

Verificación del estado académico mediante condiciones anidadas

- **Código. Programa: determinar estado académico**

```
# Programa: determinar estado académico
```

```
nota = float(input('Nota final: '))
```

```
asistencia = float(input('Porcentaje de asistencia: '))
```

```
if nota >= 3.0:
```

```
    if asistencia >= 80:
```

```
        print('Aprobado')
```

```
    else:
```

```
        print('Reprobado por inasistencia')
```

```
else:
```

```
print('Reprobado por nota')
```

El código verifica primero si la nota es igual o superior a 3,0. Si se cumple esta condición, revisa si la asistencia alcanza el 80 %; de lo contrario, informa si la reprobación fue por nota o por inasistencia.

Validación del inicio de sesión mediante condiciones anidadas

- **Código. Programa: validación de inicio de sesión**

```
# Programa: validación de inicio de sesión por niveles
```

```
usuario = input('Usuario: ')
```

```
clave = input('Contraseña: ')
```

```
if usuario == 'admin':
```

```
    if clave == '1234':
```

```
        print('Bienvenido administrador')
```

```
    else:
```

```
        print('Contraseña incorrecta')
```

```
else:
```

```
    print('Usuario no registrado')
```

El código verifica primero si el usuario ingresado es «admin» y, después, si la contraseña es «1234». Según el resultado, permite el acceso o muestra el mensaje de error correspondiente.

Aunque las estructuras anidadas son útiles, se recomienda no abusar de ellas. Un código con muchos niveles de anidamiento se vuelve difícil de leer y mantener. Esta situación se conoce como "código flecha" (arrow code), porque los bloques se van desplazando a la derecha formando una figura similar a una flecha. En muchos casos se pueden simplificar utilizando operadores lógicos (and, or), combinando las condiciones o aplicando técnicas como las cláusulas de guarda (early return).

Las estructuras anidadas son útiles para decisiones por niveles, pero no todas las situaciones requieren esa extensión. Para decisiones breves, Python ofrece el operador ternario como una alternativa compacta.

1.6. Operador ternario

Python ofrece una forma compacta de escribir estructuras condicionales simples conocida como operador ternario. Permite asignar un valor a una variable según el resultado de una condición, todo en una sola línea.

Esta forma compacta es muy valorada en la práctica profesional porque reduce líneas de código y expresa decisiones sencillas de manera directa, por ejemplo, al asignar una etiqueta, un estado o un valor por defecto. Se emplea con frecuencia dentro de mensajes, cálculos rápidos y asignaciones condicionales breves.

Conviene usarla con criterio, resulta ideal para decisiones simples, pero perjudica la legibilidad cuando la lógica es compleja o requiere varias instrucciones. Comprender su sintaxis permite decidir cuándo preferirla frente a un if-else tradicional.

- **Código. Sintaxis del operador ternario**

```
variable = valor_si_verdadero if condicion else valor_si_falso
```

```
>>> edad = 20
```

```
>>> categoria = 'mayor' if edad >= 18 else 'menor'
```

```
>>> print(categoria)
```

```
mayor
```

```
>>> numero = 7
```

```
>>> tipo = 'par' if numero % 2 == 0 else 'impar'
```

```
>>> print(tipo)
```

```
impar
```

```
>>> nota = 3.5
```

```
>>> estado = 'Aprobado' if nota >= 3.0 else 'Reprobado'
```

```
>>> print(estado)
```

```
Aprobado
```

El operador ternario también se puede usar dentro de una llamada a `print()` o como parte de una expresión más grande:

- **Código. Operador ternario dentro de `print()`**

```
>>> x = 15
```

```
>>> print(f'El número es {"par" if x % 2 == 0 else "impar"}')
```

El número es impar

Combinado con operaciones

```
>>> precio = 100
```

```
>>> total = precio * 0.9 if precio > 50 else precio
```

```
>>> print(total)
```

```
90.0
```

El operador ternario resulta muy útil para asignaciones sencillas. Sin embargo, no se recomienda cuando la lógica es compleja o cuando hay que ejecutar varias instrucciones, ya que afecta la legibilidad del código. En esos casos es preferible escribir la estructura if-else de forma tradicional.

Esta forma abreviada simplifica decisiones de dos resultados. Cuando un valor debe compararse con varios patrones definidos, la estructura match-case ofrece una organización más clara.

1.7. Estructura match-case (Python 3.10+)

A partir de la versión 3.10, Python incorporó la estructura match-case, que permite comparar un valor contra varios patrones de manera elegante. Es similar al switch-case de otros lenguajes, pero más potente, ya que admite patrones estructurales.

Esta estructura resulta especialmente útil cuando una variable debe compararse contra muchos valores fijos, como ocurre en los menús de opciones, los códigos de

estado o los comandos de una aplicación. En esos casos, match-case ofrece una alternativa más legible y organizada que una larga cadena de if-elif.

Por ser una incorporación reciente del lenguaje, solo está disponible a partir de Python 3.10, lo que conviene tener presente al elegirla. Su sintaxis se apoya en la palabra match y en bloques case que describen cada patrón a comparar.

Código. Sintaxis de la estructura match-case

```
match valor:
```

```
    case patron_1:
```

```
        bloque_1
```

```
    case patron_2:
```

```
        bloque_2
```

```
    case _:
```

```
        bloque_por_defecto
```

El guion bajo (_) actúa como comodín y equivale al bloque else. Este tipo de estructura es útil cuando se compara una variable contra varios valores fijos.

- **Código. Programa: menú con match-case**

```
# Programa: menú de opciones con match-case
```

```
opcion = input('Elija una opción (A, B, C): ').upper()
```

```
match opcion:
```

```
    case 'A':
```

```
print('Opción A seleccionada: consultar saldo')

case 'B':

    print('Opción B seleccionada: realizar retiro')

case 'C':

    print('Opción C seleccionada: hacer consignación')

case _:

    print('Opción no válida')
```

Aunque el match-case es una adición reciente al lenguaje, la estructura if-elif-else sigue siendo la más utilizada por su compatibilidad con versiones anteriores de Python y por su flexibilidad para manejar condiciones complejas con rangos u operadores lógicos.

Después de revisar las principales formas de decisión en Python, conviene compararlas para reconocer sus diferencias y seleccionar la más adecuada en cada situación.

1.8. Tipos de estructuras condicionales

Tras revisar cada estructura condicional por separado, conviene contrastarlas en conjunto para apreciar cuándo emplear cada una. La tabla comparativa las reúne indicando su sintaxis y su uso típico, lo que facilita elegir la estructura más adecuada según el tipo de decisión que deba tomar el programa.

Tabla 1. Tipos de estructuras condicionales en Python

Tipo	Sintaxis	Uso típico
if simple	if condición:	Ejecutar un bloque solo cuando la condición es verdadera.
if-else	if condición: / else:	Elegir entre dos caminos posibles.
if-elif-else	if / elif / else:	Evaluar múltiples condiciones mutuamente excluyentes.
Anidada	if dentro de if	Decisiones por niveles o combinaciones específicas.
Ternaria	valor1 if cond else valor2	Asignar un valor según una condición en una sola línea.
match-case	match valor: case patrón:	Comparar un valor contra varios patrones (Python 3.10+).

La comparación evidencia que Python ofrece una estructura para cada nivel de complejidad, desde una decisión simple hasta evaluaciones con múltiples alternativas. Reconocer estas diferencias ayuda a seleccionar la opción más legible y eficiente al resolver problemas reales de validación y control de flujo.

La comparación permite elegir una estructura según el número y el tipo de alternativas. Esta elección debe complementarse con buenas prácticas que favorezcan la claridad y el mantenimiento del código.

1.9. Buenas prácticas con estructuras condicionales

Al escribir estructuras condicionales, se recomienda seguir una serie de buenas prácticas que facilitan la legibilidad y el mantenimiento del código:

- **Usar nombres descriptivos:** asignar nombres claros a las variables facilita la comprensión de las condiciones. Por ejemplo, `es_mayor_de_edad` comunica mejor su propósito que `x`.
- **Evitar condiciones redundantes:** escribir directamente la condición hace que el código sea más claro. Por ejemplo, se recomienda usar `if edad >= 18:` en lugar de `if (edad >= 18) == True:`.
- **Limitar la profundidad del anidamiento:** se recomienda no superar tres niveles de condiciones anidadas. Si el código requiere más niveles, conviene organizarlo mediante funciones o combinar las condiciones con operadores lógicos.
- **Ordenar las condiciones por probabilidad:** en una estructura `if-elif-else`, se recomienda ubicar al comienzo las condiciones que tienen mayor probabilidad de cumplirse. Python finaliza la evaluación cuando encuentra la primera condición verdadera.
- **Utilizar el bloque `else` con prudencia:** este bloque se emplea cuando debe ejecutarse una acción si ninguna condición anterior se cumple. Si solo se necesita actuar ante una condición específica, basta con utilizar `if`.

Aplicar estas prácticas desde el inicio del aprendizaje crea hábitos sólidos que se reflejan en códigos más limpios, más fáciles de depurar y más colaborativos, cualidades esenciales en entornos profesionales de desarrollo. Para construir las expresiones que

orientan las decisiones del programa, a continuación, se presentan los operadores condicionales.

2. Operadores condicionales

Los operadores condicionales son símbolos o palabras reservadas que permiten construir las expresiones que evalúan las estructuras condicionales. Son elementos fundamentales del lenguaje porque posibilitan comparar valores, combinar condiciones, verificar pertenencia y asignar datos a las variables. Un dominio adecuado de los operadores es indispensable para construir expresiones claras y correctas, que reflejen con precisión la lógica del problema que se desea resolver.

Para reconocer su función dentro del código, los operadores abordados en este componente se organizan en cinco grupos:

- **Relacionales:** comparan dos valores.
- **Lógicos:** combinan expresiones booleanas.
- **De inclusión:** verifican la pertenencia de un elemento.
- **De asignación:** almacenan o actualizan valores.
- **De identidad:** comprueban si dos referencias corresponden al mismo objeto.

Esta clasificación facilita la elección del operador adecuado según la comparación, validación o actualización que requiera el programa.

Los operadores se agrupan según la función que cumplen dentro de una expresión. El recorrido comienza con los operadores relacionales, empleados para comparar valores.

2.1. Operadores relacionales

Los operadores relacionales comparan dos valores y devuelven un resultado booleano: True si la comparación se cumple y False en caso contrario. Son la base de las condiciones que se evalúan en las estructuras if, elif, while y otros contextos donde se necesita tomar una decisión. Python admite aplicar estos operadores a datos numéricos, cadenas de caracteres, listas, tuplas y cualquier tipo que defina reglas de comparación.

Para aplicar correctamente las comparaciones dentro de una condición, es necesario conocer cada operador relacional y el resultado que produce. La siguiente tabla recoge cada operador con un ejemplo y su valor de retorno, de modo que sirva como referencia rápida al construir expresiones booleanas.

Tabla 2. Operadores relacionales en Python

Operador	Nombre	Ejemplo	Resultado
==	Igual a	5 == 5	True
!=	Diferente de	5 != 3	True
>	Mayor que	5 > 3	True
<	Menor que	5 < 3	False
>=	Mayor o igual	5 >= 5	True
<=	Menor o igual	3 <= 5	True

Estos operadores constituyen la base de toda condición evaluada en las estructuras if, elif y while.

Dominarlos permite traducir con precisión las reglas de un problema a expresiones que el programa pueda evaluar, paso indispensable para la toma de decisiones.

Para observar cómo los operadores relacionales comparan valores y generan resultados booleanos, a continuación, se presentan ejemplos aplicados con edades, rangos numéricos y validación de datos:

- **Código. Operadores relacionales**

Ejemplos de operadores relacionales

```
>>> edad = 25
```

```
>>> edad == 25    # True
```

```
>>> edad != 30    # True
```

```
>>> edad > 18     # True
```

```
>>> edad < 18     # False
```

```
>>> edad >= 25    # True
```

```
>>> edad <= 20    # False
```

El código compara la edad, cuyo valor es 25, mediante distintos operadores relacionales. Cada expresión devuelve True o False, según se cumpla o no la comparación.

- **Código. Comparación de cadenas**

Comparación de cadenas

```
>>> 'ana' < 'beto'    # True (compara alfabéticamente)
```

```
>>> 'Python' == 'python' # False (distingue mayúsculas)
```

```
>>> 'Hola' != 'Adiós'   # True
```

```
>>> 'A' < 'a'          # True (65 < 97 en Unicode)
```

```
>>> 'Z' < 'a'          # True (90 < 97 en Unicode)
```

El código compara palabras y letras teniendo en cuenta el orden Unicode y la diferencia entre mayúsculas y minúsculas. Por esta razón, 'Python' y 'python' se consideran cadenas diferentes.

- **Código. Encadenamiento de operadores relacionales**

```
# Encadenamiento de operadores relacionales
```

```
>>> edad = 25
```

```
>>> 18 <= edad <= 65    # True
```

```
>>> 0 < 7 < 10          # True
```

```
# Equivale a escribir:
```

```
>>> 18 <= edad and edad <= 65 # True
```

El código verifica si un valor se encuentra entre dos límites mediante una sola expresión. La comparación `18 <= edad <= 65` equivale a combinar las dos condiciones con el operador `and`.

- **Código. Programa: validar datos de registro**

```
# Programa: validar datos de registro
```

```
nombre = input('Nombre: ')

edad = int(input('Edad: '))

if len(nombre) == 0:

    print('Error: el nombre no puede estar vacío')

elif edad < 0:

    print('Error: la edad no puede ser negativa')

elif edad > 150:

    print('Error: la edad parece incorrecta')

else:

    print(f'Bienvenido, {nombre}')
```

El código comprueba que el nombre no esté vacío y que la edad se encuentre entre 0 y 150. Si los datos son válidos, muestra un mensaje de bienvenida; de lo contrario, informa el error encontrado.

Los operadores relacionales permiten comparar números, cadenas de caracteres y otros datos compatibles. En las cadenas, Python aplica el orden Unicode y distingue entre mayúsculas y minúsculas, aspecto que debe considerarse al interpretar los resultados.

Las comparaciones anteriores generan resultados booleanos que permiten evaluar una condición. Cuando una decisión depende de dos o más condiciones, estos resultados se combinan mediante operadores lógicos, tema que se presenta a continuación.

2.2. Operadores lógicos

Los operadores lógicos permiten combinar varias expresiones booleanas en una sola condición. Python dispone de tres operadores lógicos: `and`, `or` y `not`. Estos operadores son fundamentales cuando una decisión depende de varias condiciones relacionadas. A diferencia de otros lenguajes que utilizan símbolos como `&&`, `||` o `!`, Python emplea las palabras en inglés, lo que hace el código más legible.

Cuando una decisión depende de más de una condición, los operadores lógicos permiten combinarlas en una sola expresión. La tabla describe los tres operadores disponibles en Python (`and`, `or` y `not`) con ejemplos que ilustran su resultado.

Tabla 3. Operadores lógicos en Python.

Operador	Descripción	Ejemplo	Resultado
<code>and</code>	Verdadero si ambas expresiones son verdaderas.	<code>(5 > 3) and (2 < 4)</code>	True
<code>or</code>	Verdadero si al menos una expresión es verdadera.	<code>(5 > 3) or (2 > 4)</code>	True
<code>not</code>	Invierte el valor lógico de la expresión.	<code>not (5 > 3)</code>	False

Combinar condiciones con and, or y not amplía la capacidad expresiva del programa y evita escribir múltiples estructuras anidadas. Su uso correcto es clave para reflejar fielmente la lógica del problema, especialmente en validaciones que dependen de varios criterios simultáneos.

Para anticipar el resultado de una condición compuesta, resulta útil observar el comportamiento de los operadores lógicos ante todas las combinaciones posibles de valores. La tabla de verdad presenta esos resultados de forma sistemática.

Tabla 4. Tabla de verdad de los operadores lógicos.

A	B	A and B	A or B	not A
True	True	True	True	False
True	False	False	True	False
False	True	False	True	True
False	False	False	False	True

La tabla de verdad permite predecir con certeza el valor de cualquier expresión lógica antes de ejecutarla, lo que ayuda a depurar condiciones y a evitar errores difíciles de detectar. Es una herramienta de referencia que el programador consulta al diseñar validaciones complejas.

Para comprender cómo los operadores and, or y not intervienen en la toma de decisiones, a continuación, se presentan ejemplos relacionados con el acceso a un sistema, los días de atención y la aprobación de un crédito:

- **Código. Uso de operadores lógicos: and, or, not**

Ejemplo: validar acceso a un sistema

```
edad = 25
```

```
tiene_cuenta = True
```

```
if edad >= 18 and tiene_cuenta:
```

```
    print('Acceso permitido')
```

```
else:
```

```
    print('Acceso denegado')
```

El código permite el acceso porque la persona tiene 25 años y posee una cuenta activa. El operador and exige que ambas condiciones sean verdaderas.

- **Código. Operador or: verificar día de atención**

Ejemplo: verificar día de atención

```
dia = 'sábado'
```

```
if dia == 'sábado' or dia == 'domingo':
```

```
    print('Fin de semana')
```

```
else:
```

```
    print('Día hábil')
```

Ejemplo: operador not

```
esta_lloviendo = False
```

```
if not esta_lloviendo:
```

```
    print('Puede salir sin paraguas')
```

El operador `or` identifica el sábado o el domingo como fin de semana. Por su parte, `not` invierte el valor `False` y permite mostrar que se puede salir sin paraguas.

- **Código. Combinación de operadores lógicos**

```
# Ejemplo: condición compleja
```

```
edad = 30
```

```
salario = 2500000
```

```
tiene_codeudor = False
```

```
# Aprueba crédito si:
```

```
# - Tiene más de 25 años Y gana más de 2 millones, O
```

```
# - Tiene codeudor
```

```
aprobado = (edad > 25 and salario > 2000000) or tiene_codeudor
```

```
if aprobado:
```

```
    print('Crédito aprobado')
```

```
else:
```

```
    print('Crédito rechazado')
```

El código aprueba el crédito si la persona tiene más de 25 años y gana más de \$2.000.000, o si cuenta con codeudor. Si ninguna de estas condiciones se cumple, el crédito se rechaza.

Los operadores lógicos articulan varias condiciones en una sola expresión. Si la validación consiste en comprobar si un elemento pertenece a una colección, se emplean los operadores de inclusión.

2.3. Operadores de inclusión

Los operadores de inclusión (también llamados de pertenencia) verifican si un elemento se encuentra dentro de una secuencia, como una cadena, una lista, una tupla, un conjunto o un diccionario. Python ofrece dos operadores de inclusión: `in` y `not in`. Son muy útiles cuando se necesita validar si un dato pertenece a un grupo de valores permitidos, sin tener que escribir múltiples comparaciones con el operador `or`.

Tabla 5. Operadores de inclusión en Python.

Operador	Descripción	Ejemplo	Resultado
<code>in</code>	Verdadero si el elemento está presente en la secuencia.	<code>'a' in 'hola'</code>	True
<code>not in</code>	Verdadero si el elemento no está presente en la secuencia.	<code>5 not in [1, 2, 3]</code>	True

Los operadores de inclusión simplifican las validaciones de pertenencia y sustituyen largas cadenas de comparaciones con `or`, haciendo el código más legible. Son especialmente prácticos al comprobar opciones de menús, listas de valores permitidos o caracteres dentro de una cadena.

Para comprender cómo se verifica la pertenencia de un dato en diferentes colecciones, se presentan ejemplos con cadenas, listas, tuplas y diccionarios, además de aplicaciones para simplificar comparaciones y validar opciones:

- **Código. Operadores de inclusión en cadenas y listas**

```
# Verificar pertenencia en cadenas
```

```
>>> 'Python' in 'Aprende Python hoy'
```

```
True
```

```
>>> 'Java' not in 'Aprende Python hoy'
```

```
True
```

```
# Verificar pertenencia en listas
```

```
>>> frutas = ['manzana', 'pera', 'uva']
```

```
>>> 'pera' in frutas
```

```
True
```

```
>>> 'mango' not in frutas
```

```
True
```

```
# Verificar pertenencia en tuplas
```

```
>>> colores = ('rojo', 'verde', 'azul')
```

```
>>> 'verde' in colores
```

```
True
```

```
# Verificar pertenencia en diccionarios (revisa las claves)
```

```
>>> notas = {'María': 4.5, 'Juan': 3.8}
```

```
>>> 'María' in notas
```

```
True
```

```
>>> 4.5 in notas      # False: busca en claves, no en valores
```

```
False
```

```
>>> 4.5 in notas.values() # True
```

```
True
```

El código utiliza `in` y `not in` para verificar elementos en cadenas, listas, tuplas y diccionarios. En los diccionarios, `in` busca en las claves; para consultar los valores, se utiliza `values()`.

- **Código. Comparación: forma larga vs. operador in**

```
# Forma larga (no recomendada)
```

```
if x == 'a' or x == 'b' or x == 'c' or x == 'd':
```

```
    print('Letra válida')
```

```
# Forma corta con operador in (recomendada)
```

```
if x in ['a', 'b', 'c', 'd']:
```

```
    print('Letra válida')
```

El código muestra dos formas de comprobar si una letra pertenece a un grupo de opciones. El operador in reemplaza varias comparaciones con or y facilita la lectura de la condición.

- **Código. Programa: validar día de la semana**

```
# Ejemplo: validar día de la semana
```

```
dia = input('Ingrese el día: ').lower()
```

```
dias_validos = ['lunes', 'martes', 'miércoles',
```

```
                'jueves', 'viernes', 'sábado', 'domingo']
```

```
if dia in dias_validos:
```

```
    print(f'{dia} es un día válido')
```

```
else:
```

```
    print('Día no reconocido')
```

```
# Ejemplo: verificar si un día es laboral
```

```
fin_semana = ['sábado', 'domingo']
```

```
if dia not in fin_semana:
```

```
    print('Es un día laboral')
```

else:

```
print('Es fin de semana')
```

El código comprueba si el dato ingresado corresponde a un día válido. Después, determina si se trata de un día laboral o de fin de semana.

La pertenencia permite validar si un dato se encuentra o no en una secuencia. El siguiente subtema presenta los operadores de asignación, utilizados para almacenar y actualizar valores en variables.

2.4. Operadores de asignación

Los operadores de asignación se utilizan para almacenar un valor en una variable. El operador básico es =, que asigna el valor de la derecha a la variable de la izquierda. Python también ofrece operadores de asignación compuesta que combinan una operación aritmética con la asignación, lo que permite escribir código más conciso y expresivo.

Además del operador básico de asignación, Python ofrece variantes que combinan una operación con la asignación en un solo paso. La tabla resume las variantes indicando a qué expresión equivale cada una y un ejemplo de uso.

Tabla 6. Operadores de asignación en Python.

Operador	Descripción	Equivale a	Ejemplo
=	Asignación simple	-	x = 10
+=	Asignación con suma	x = x + 5	x += 5

Operador	Descripción	Equivale a	Ejemplo
<code>--</code>	Asignación con resta	$x = x - 3$	<code>x -= 3</code>
<code>*=</code>	Asignación con multiplicación	$x = x * 2$	<code>x *= 2</code>
<code>/=</code>	Asignación con división	$x = x / 4$	<code>x /= 4</code>
<code>//=</code>	Asignación con división entera	$x = x // 2$	<code>x //= 2</code>
<code>%=</code>	Asignación con módulo	$x = x \% 3$	<code>x %= 3</code>
<code>**=</code>	Asignación con potencia	$x = x ** 2$	<code>x **= 2</code>

La asignación compuesta produce un código más conciso y expresivo, sobre todo en contadores y acumuladores dentro de los ciclos. Familiarizarse con estas formas abreviadas facilita la lectura de programas y es una práctica habitual entre desarrolladores.

La asignación compuesta facilita la actualización de variables mediante expresiones breves y claras. Para comprender su aplicación con valores numéricos, cadenas, listas y asignaciones múltiples, a continuación, se presentan algunos ejemplos:

- **Código. Operadores de asignación compuesta**

Ejemplos de operadores de asignación

```
>>> x = 10
```

```
>>> x += 5    # x = 15
```

```
>>> x -= 3    # x = 12
```

```
>>> x *= 2    # x = 24
```

```
>>> x /= 4    # x = 6.0
```

```
>>> x //= 2    # x = 3.0
```

```
>>> x **= 2    # x = 9.0
```

El código inicia la variable x con el valor 10 y la actualiza mediante operaciones de suma, resta, multiplicación, división y potencia. Cada operador realiza el cálculo y almacena el nuevo resultado en la misma variable.

- **Código. Asignación compuesta con cadenas y listas**

Operador += con cadenas

```
>>> mensaje = 'Hola'
```

```
>>> mensaje += ' mundo'
```

```
>>> print(mensaje)
```

Hola mundo

Operador += con listas

```
>>> numeros = [1, 2, 3]
```

```
>>> numeros += [4, 5]
```

```
>>> print(numeros)
```

```
[1, 2, 3, 4, 5]
```

```
# Operador *= con cadenas
```

```
>>> linea = '-'
```

```
>>> linea *= 20
```

```
>>> print(linea)
```

```
-----
```

El código utiliza += para unir cadenas y agregar elementos a una lista. También emplea *= para repetir una cadena según la cantidad indicada.

- **Código. Asignación múltiple e intercambio**

```
# Asignación múltiple
```

```
>>> a, b, c = 1, 2, 3
```

```
>>> print(a, b, c)
```

```
1 2 3
```

```
# Intercambio de valores (swap)
```

```
>>> a, b = b, a
```

```
>>> print(a, b)
```

```
2 1
```

```
# Asignación encadenada
```

```
>>> x = y = z = 0
```



```
>>> print(x, y, z)
```

```
0 0 0
```

El código asigna varios valores en una sola línea, intercambia el contenido de dos variables y establece un mismo valor para varias variables. Estas formas reducen instrucciones y facilitan la lectura del código.

Los operadores de asignación modifican el contenido de una variable. En contraste, los operadores de identidad permiten comprobar si dos referencias corresponden al mismo objeto en memoria.

2.5. Operadores de identidad

Los operadores de identidad comparan si dos variables apuntan al mismo objeto en memoria. Python ofrece dos operadores de identidad: `is` e `is not`. A diferencia del operador `==` (que compara valores), los operadores de identidad verifican si dos referencias apuntan exactamente al mismo objeto.

Comparar valores no es lo mismo que comparar objetos en memoria, distinción que en Python se expresa con los operadores de identidad. La tabla presenta `is` e `is not` con ejemplos que aclaran su comportamiento.

Tabla 7. Operadores de identidad en Python.

Operador	Descripción	Ejemplo
<code>is</code>	Verdadero si ambas variables son el mismo objeto.	<code>x is None</code>
<code>is not</code>	Verdadero si las variables son objetos diferentes.	<code>x is not None</code>

Distinguir entre igualdad de valor (==) e identidad de objeto (is) evita errores sutiles, en especial al comparar con None. Comprender esta diferencia fortalece el manejo correcto de referencias y mejora la calidad del código.

- **Código. Diferencia entre == (valor) e is (identidad)**

```
# Diferencia entre == e is
```

```
>>> a = [1, 2, 3]
```

```
>>> b = [1, 2, 3]
```

```
>>> a == b    # True: tienen los mismos valores
```

```
>>> a is b     # False: son objetos diferentes
```

```
>>> c = a
```

```
>>> c is a     # True: son el mismo objeto
```

```
# Uso recomendado con None
```

```
>>> valor = None
```

```
>>> if valor is None:
```

```
...     print('El valor no existe')
```

```
El valor no existe
```

La comparación con None siempre debe hacerse con is en lugar de ==. Esto se debe a que None es un objeto único en Python (un singleton), y el operador is es más rápido y semánticamente correcto para este caso. Esta regla forma parte de las recomendaciones del estándar PEP 8.

Diferenciar identidad e igualdad evita comparaciones incorrectas. Cuando una expresión reúne varios operadores, también es necesario conocer el orden en que Python los evalúa.

2.6. Precedencia de los operadores condicionales

Cuando una expresión combina varios operadores, Python los evalúa según una jerarquía llamada precedencia. Los operadores con mayor precedencia se evalúan primero. En el contexto de las estructuras condicionales, la jerarquía general es la siguiente:

- a) Operadores aritméticos.
- b) Operadores relacionales.
- c) Operador lógico not.
- d) Operador lógico and.
- e) Operador lógico or.

Cuando una expresión combina varios operadores, el orden en que Python los evalúa determina el resultado. La tabla organiza esa jerarquía por niveles, de mayor a menor precedencia, como guía para interpretar expresiones complejas.

Tabla 8. Precedencia de operadores en Python

Nivel	Tipo de operador	Operadores
1 (mayor)	Aritméticos de potencia	**
2	Aritméticos multiplicativos	*, /, //, %
3	Aritméticos aditivos	+, -

Nivel	Tipo de operador	Operadores
4	Relacionales	==, !=, >, <, >=, <=
5	Inclusión e identidad	in, not in, is, is not
6	Lógico not	not
7	Lógico and	and
8 (menor)	Lógico or	or

Conocer la precedencia permite predecir el resultado de una expresión y emplear paréntesis para hacerla más clara cuando sea necesario. Respetar esta jerarquía previene errores lógicos y da lugar a condiciones más predecibles y fáciles de mantener.

- **Código. Precedencia de operadores condicionales**

Ejemplo de precedencia

```
>>> resultado = 2 + 3 * 4 > 10 and not False
```

```
>>> # Evaluación paso a paso:
```

```
>>> # 1) 3 * 4 = 12
```

```
>>> # 2) 2 + 12 = 14
```

```
>>> # 3) 14 > 10 = True
```

```
>>> # 4) not False = True
```

```
>>> # 5) True and True = True
```

```
>>> print(resultado)
```

True

Con paréntesis para mayor claridad

```
>>> resultado = ((2 + 3 * 4) > 10) and (not False)
```

Aunque Python respeta la precedencia automáticamente, se recomienda usar paréntesis cuando una expresión combina varios operadores. Esto mejora la legibilidad y evita confusiones, especialmente en expresiones complejas donde el orden de evaluación podría no ser obvio para quien lea el código por primera vez.

La precedencia permite interpretar condiciones complejas de manera correcta. A partir de esta base, se estudian las estructuras iterativas, que repiten instrucciones y procesan conjuntos de datos.

Para fortalecer la comprensión sobre el uso de los operadores en Python, se presenta el siguiente pódcast instruccional. Este recurso explica, mediante ejemplos sencillos, cómo comparar datos, combinar condiciones, verificar pertenencias, actualizar valores, comprobar la identidad de los objetos y reconocer el orden de evaluación de una expresión.

Transcripción del pódcast. Operadores que activan la lógica en Python
Mujer: bienvenidos al pódcast “Operadores que activan la lógica en Python”. En este episodio explicaremos, de forma clara y sencilla, cómo los operadores permiten que un programa compare información, valide condiciones y tome decisiones.

Hombre: así es. En Python no basta con ejecutar instrucciones una tras otra. Muchas veces el programa necesita comparar datos, verificar condiciones o comprobar si cierta información cumple determinados criterios antes de continuar.

Mujer: por eso, en este episodio conoceremos los principales operadores de Python y veremos cómo permiten construir la lógica que utilizan los programas para tomar decisiones de forma clara y organizada. ¡Comencemos!

Mujer: empecemos con los operadores relacionales. Su función es comparar dos valores para determinar si una condición se cumple. Permiten verificar si dos datos son iguales, diferentes, mayores, menores, mayores o iguales, o menores o iguales.

Hombre: el resultado de estas comparaciones siempre será un valor lógico: True, cuando la condición es verdadera, o False, cuando no se cumple. Estas respuestas son la base para que un programa pueda decidir qué acción ejecutar.

Mujer: después aparecen los operadores lógicos, que permiten combinar varias condiciones. Python utiliza and, or y not. El operador and se usa cuando dos condiciones deben cumplirse al mismo tiempo; el operador or se aplica cuando basta con que se cumpla una condición. El operador not invierte el resultado lógico de una expresión.

Hombre: otro grupo importante son los operadores de inclusión, in y not in. Estos permiten verificar si un dato pertenece o no a una secuencia. Son útiles para revisar listas, cadenas, tuplas, conjuntos o diccionarios. Por ejemplo, permiten comprobar si una opción ingresada está dentro de una lista de respuestas válidas.

Mujer: también encontramos los operadores de asignación, que permiten guardar o actualizar valores en las variables. Además del signo igual =, que se lee como “asignar” o “guardar un valor”, Python ofrece formas abreviadas como +=, que significa “sumar y asignar”; -=, “restar y asignar”; *=, “multiplicar y asignar”; /=, “dividir y asignar”; y %=, “calcular el residuo y asignar”.

Hombre: estas expresiones facilitan el trabajo con contadores, acumuladores y ciclos, porque permiten modificar el valor de una variable de forma más corta y ordenada.

Mujer: otro concepto importante corresponde a los operadores de identidad, is e is not. Estos permiten comprobar si dos variables hacen referencia al mismo elemento en la memoria del programa.

Hombre: en palabras sencillas, ayudan a saber si dos nombres apuntan exactamente al mismo objeto. También se usan para revisar si una variable tiene el valor especial None, que significa que no tiene ningún dato asignado.

Mujer: finalmente, la precedencia de operadores indica el orden en que Python evalúa una expresión. Cuando una condición combina varios operadores, el lenguaje sigue una jerarquía.

Hombre: y cuando una expresión resulta extensa o puede generar dudas, una buena práctica consiste en utilizar paréntesis para hacer más clara la lógica del programa y facilitar su lectura.

Mujer: en conclusión, los operadores activan la lógica de Python porque permiten comparar, validar, combinar y organizar decisiones dentro de un programa.

Hombre: su uso adecuado ayuda a escribir instrucciones más claras, seguras y fáciles de revisar. Gracias por acompañarnos en este episodio ¡Hasta la próxima!

3. Estructuras iterativas: concepto, tipos, sintaxis, sentencias

Las estructuras iterativas, también conocidas como ciclos o bucles, permiten ejecutar un bloque de instrucciones varias veces mientras se cumpla una condición determinada. Son fundamentales en la programación porque evitan la repetición manual de código y facilitan el procesamiento de grandes volúmenes de datos.

Las estructuras iterativas evitan repetir instrucciones de forma manual. Gracias a ellas, un programa puede procesar grandes cantidades de datos de manera organizada y eficiente.

Python dispone de dos estructuras iterativas principales y de sentencias que permiten controlar su ejecución:

- a) Ciclo while:** repite un bloque mientras una condición sea verdadera.
- b) Ciclo for:** recorre los elementos de una secuencia.
- c) Sentencias de control:** break, continue y else modifican el desarrollo del ciclo.

Cada estructura responde a una necesidad específica. Elegir la opción adecuada permite crear programas más claros, organizados y eficientes.

Las estructuras responden a distintas necesidades de repetición. Primero se presenta el ciclo while, adecuado cuando la ejecución depende de una condición y no se conoce previamente el número de iteraciones.

3.1. Ciclo while

El ciclo while repite un bloque de instrucciones mientras una condición sea verdadera. Antes de cada repetición, Python evalúa la condición: si se cumple, ejecuta el bloque; de lo contrario, finaliza el ciclo y continúa con la siguiente instrucción.

Esta estructura se compone de la palabra reservada while, una condición y un bloque indentado. La relación entre estos elementos permite identificar qué instrucciones se repiten y cuándo finaliza el ciclo.

Para comprender su funcionamiento, a continuación, se presentan cinco códigos que muestran la sintaxis general del ciclo while y su aplicación para contar valores, controlar un menú, validar datos y acumular números:

- **Código. Sintaxis general del ciclo while**

```
while condicion:
```

```
    instrucción_1
```

```
    instrucción_2
```

```
    ...
```

```
    instrucción_n
```

El ciclo while evalúa una condición antes de cada repetición y ejecuta el bloque indentado mientras sea verdadera. Cuando la condición es falsa, el ciclo termina y el programa continúa.

- **Código. Programa: contar del 1 al 5 con while**

```
# Programa: contar del 1 al 5
```

```
contador = 1

while contador <= 5:

    print(contador)

    contador += 1

print('Fin del ciclo')
```

El código muestra los números del 1 al 5 y aumenta el contador en cada repetición. Cuando el contador llega a 6, la condición deja de cumplirse y aparece el mensaje “Fin del ciclo”.

- **Código. Programa: menú interactivo con while**

```
# Programa: menú de opciones con ciclo while

opcion = ""

while opcion != '4':

    print('\n===== MENÚ =====')

    print('1. Consultar saldo')

    print('2. Realizar retiro')

    print('3. Hacer consignación')

    print('4. Salir')

    opcion = input('Elija una opción: ')
```

```
if opcion == '1':  
    print('Su saldo es $500.000')  
  
elif opcion == '2':  
    print('Retiro realizado con éxito')  
  
elif opcion == '3':  
    print('Consignación exitosa')  
  
elif opcion == '4':  
    print('Saliendo del sistema...')  
  
else:  
    print('Opción no válida')
```

El código mantiene activo el menú y ejecuta una acción según la opción seleccionada. La repetición termina cuando se elige la opción 4, correspondiente a salir del sistema.

- **Código. Programa: validar entrada de datos con while**

```
# Programa: validar entrada de edad  
  
edad = -1  
  
while edad < 0 or edad > 150:  
    edad = int(input('Ingrese su edad (0-150): '))  
  
if edad < 0 or edad > 150:
```

```
print('Edad no válida, intente de nuevo')
```

```
print(f'Edad registrada: {edad}')
```

El código solicita nuevamente la edad mientras el valor sea menor que 0 o mayor que 150. Cuando se ingresa un dato válido, finaliza el ciclo y muestra la edad registrada.

- **Código. Programa: sumar números con acumulador**

```
# Programa: sumar números hasta que el usuario quiera terminar
```

```
continuar = True
```

```
suma = 0
```

```
while continuar:
```

```
    numero = float(input('Ingrese un número: '))
```

```
    suma += numero
```

```
    respuesta = input('¿Ingresar otro? (s/n): ').lower()
```

```
    if respuesta != 's':
```

```
        continuar = False
```

```
print(f'Suma total: {suma}')
```

El código solicita números y acumula cada valor en la variable suma. El ciclo continúa mientras se responda “s” y, al finalizar, muestra el total obtenido.

El ciclo while mantiene la repetición mientras una condición sea verdadera. Cuando se requiere recorrer una secuencia o un conjunto de valores, el ciclo for ofrece una estructura más directa.

3.2. Ciclo for

Es utilizado para recorrer los elementos de una secuencia, como una lista, una tupla, una cadena de caracteres o un rango de números. En cada iteración, Python asigna automáticamente el siguiente elemento de la secuencia a una variable de control y ejecuta el bloque de instrucciones.

El ciclo for se utiliza con frecuencia para procesar colecciones de datos en tareas como las siguientes:

- Recorrer los productos de un inventario.
- Calcular el promedio de una lista de notas.
- Procesar los caracteres de una cadena.

Estas aplicaciones se resuelven de manera clara y directa, por lo que este ciclo es una herramienta fundamental para el análisis de datos.

Su estructura es directa, la palabra for, una variable que recibe cada elemento, la palabra in y la secuencia que se recorre, seguidas de un bloque indentado. Identificar estas partes facilita leer cualquier recorrido, ya sea sobre listas, cadenas o rangos numéricos.

Para reconocer las principales aplicaciones del ciclo for, a continuación, se presenta una infografía interactiva con ejemplos de recorridos en listas, cadenas, tuplas, diccionarios y rangos numéricos.

A. Recorridos y aplicaciones del ciclo for

- **Funcionamiento general**

El ciclo for recorre una secuencia y procesa un elemento en cada repetición. Por ejemplo, puede revisar cinco productos hasta completar la lista.

- **Recorrido de listas**

Una lista almacena elementos en un orden determinado. Por ejemplo, puede recorrer una lista para mostrar los nombres de varias frutas.

- **Recorrido de cadenas**

Una cadena está formada por una secuencia de caracteres. Por ejemplo, al procesar la palabra «Python», identifica cada letra desde la «P» hasta la «n».

- **Recorrido de tuplas**

Una tupla agrupa datos ordenados que permanecen sin modificaciones. Por ejemplo, puede consultar por separado los valores de unas coordenadas.

- **Recorrido de diccionarios**

Un diccionario organiza información mediante claves y valores relacionados. Por ejemplo, puede asociar el nombre de cada estudiante con su calificación.

- **Repeticiones con rangos**

La función `range()` genera una secuencia numérica para controlar las repeticiones. Por ejemplo, un rango del 1 al 10 permite construir una tabla de multiplicar.

- **Procesamiento de datos**

El ciclo realiza una misma operación con todos los valores de una colección. Por ejemplo, puede sumar varias calificaciones para calcular su promedio.

Los ejemplos muestran que el ciclo `for` permite recorrer diferentes secuencias y procesar sus elementos de forma ordenada. Su aplicación facilita la automatización de tareas repetitivas y mejora la organización del programa.

Los ciclos `while` y `for` permiten repetir instrucciones, aunque responden a situaciones diferentes. La siguiente comparación facilita reconocer cuándo conviene utilizar cada uno.

3.3. Sentencias de control: `break`, `continue` y `else`

Python ofrece tres sentencias especiales para modificar el flujo normal de los ciclos: `break`, `continue` y `else`. Estas sentencias permiten construir ciclos más flexibles y responder a situaciones particulares durante la ejecución. Su uso adecuado simplifica la lógica y reduce la necesidad de estructuras anidadas complejas.

La sentencia `break` interrumpe inmediatamente el ciclo, sin importar si la condición sigue siendo verdadera. Se utiliza cuando se detecta una situación que exige

salir del ciclo de forma anticipada, como encontrar un elemento buscado o detectar un error crítico.

- **# Ejemplo de break: buscar un número**

```
numeros = [1, 5, 8, 12, 17, 20]
```

```
for num in numeros:
```

```
    if num > 10:
```

```
        print(f'Primer número mayor a 10: {num}')
```

```
        break
```

```
# Salida: Primer número mayor a 10: 12
```

```
# Ejemplo de break en un while: adivinar número
```

```
import random
```

```
numero_secreto = random.randint(1, 10)
```

```
intentos = 0
```

```
while True:
```

```
    intento = int(input('Adivine el número (1-10): '))
```

```
    intentos += 1
```

```
    if intento == numero_secreto:
```

```
        print(f'¡Correcto! Lo logró en {intentos} intentos')
```

```
        break
```

```
elif intento < numero_secreto:

    print('Muy bajo, intente un número mayor')

else:

    print('Muy alto, intente un número menor')
```

La sentencia continue omite las instrucciones restantes de la iteración actual y pasa a la siguiente. A diferencia de break, no termina el ciclo, solo salta una iteración. Es útil cuando ciertos elementos deben ser ignorados sin detener el procesamiento.

- **# Ejemplo de continue: imprimir solo pares**

```
for num in range(1, 11):

    if num % 2 != 0:

        continue

    print(num)

# Salida: 2, 4, 6, 8, 10

# Ejemplo de continue: procesar solo datos válidos

datos = [10, -5, 8, 0, 'dato', 15, None]

suma = 0

for valor in datos:

    # Saltar valores no numéricos o no positivos

    if not isinstance(valor, (int, float)) or valor <= 0:
```

```
        continue

    suma += valor

print(f'Suma de valores positivos: {suma}')

# Salida: Suma de valores positivos: 33
```

La cláusula else se puede asociar tanto a while como a for. Se ejecuta solo cuando el ciclo termina de manera natural (porque la condición dejó de cumplirse o se recorrió toda la secuencia). Si el ciclo se interrumpe con break, el bloque else no se ejecuta. Esta característica es útil para identificar si un ciclo completó su recorrido sin encontrar una condición de salida.

- **# Ejemplo de else en un ciclo**

```
for num in range(1, 6):

    print(num)

else:

    print('Ciclo terminado correctamente')

# Salida: 1, 2, 3, 4, 5, Ciclo terminado correctamente

# Ejemplo: verificar si un número es primo

numero = 17

for i in range(2, numero):

    if numero % i == 0:

        print(f'{numero} no es primo, divisible por {i}')
```

```
        break

    else:

        print(f'{numero} es un número primo')

# Salida: 17 es un número primo
```

Las sentencias de control permiten interrumpir, omitir o completar iteraciones. Cuando una repetición debe ejecutarse dentro de otra, se utilizan ciclos anidados.

3.4. Ciclos anidados

Un ciclo anidado contiene otro ciclo en su interior. El ciclo externo controla el recorrido principal, mientras que el interno completa sus repeticiones cada vez que avanza el ciclo exterior. Esta estructura se utiliza para procesar información organizada en dos dimensiones, como matrices y tablas.

El anidamiento también permite recorrer filas y columnas de una hoja de cálculo, analizar los píxeles de una imagen, generar combinaciones y comparar los elementos de dos listas. Su aplicación facilita el procesamiento de datos que requieren varios niveles de repetición.

Para comprender cómo funcionan los ciclos anidados, a continuación, se presentan tres ejemplos aplicados: la generación de una tabla de multiplicar, la construcción de un triángulo de asteriscos y la búsqueda de coincidencias entre dos listas:

- **Código. Ciclos anidados: tabla de multiplicar**

Ejemplo: generar una tabla de multiplicar

```
for i in range(1, 4):  
  
    for j in range(1, 4):  
  
        producto = i * j  
  
        print(f'{i} x {j} = {producto}')  
  
    print('---')
```

Salida:

```
# 1 x 1 = 1, 1 x 2 = 2, 1 x 3 = 3, ---  
  
# 2 x 1 = 2, 2 x 2 = 4, 2 x 3 = 6, ---  
  
# 3 x 1 = 3, 3 x 2 = 6, 3 x 3 = 9, ---
```

El ciclo externo recorre los números del 1 al 3 y el interno los combina con el mismo rango. En cada repetición, se calcula y se muestra el producto correspondiente.

- **Código. Programa: triángulo de asteriscos**

Programa: triángulo de asteriscos

```
filas = 5  
  
for i in range(1, filas + 1):  
  
    for j in range(i):  
  
        print('*', end='')  
  
    print()
```

```
print() # Salto de línea

# Salida:

# *

# **

# ***

# ****

# *****
```

El ciclo externo determina las cinco filas del triángulo y el interno imprime los asteriscos de cada una. La cantidad aumenta en cada fila hasta completar la figura.

- **Código. Programa: buscar coincidencias entre listas**

```
# Ejemplo: buscar coincidencias entre dos listas

lista_a = [1, 3, 5, 7, 9]

lista_b = [2, 4, 6, 7, 10]

coincidencias = []

for elemento_a in lista_a:

    for elemento_b in lista_b:

        if elemento_a == elemento_b:

            coincidencias.append(elemento_a)

print(f'Elementos comunes: {coincidencias}')
```

Salida: Elementos comunes: [7]

Los ciclos comparan cada elemento de la primera lista con todos los elementos de la segunda. Cuando encuentran un valor común, lo agregan a la lista de coincidencias; en este caso, el resultado es 7.

Estos ciclos son potentes, pero aumentan el costo computacional. Si el ciclo externo realiza n iteraciones y el interno también, el número total de ejecuciones será $n \times n$. Por esta razón, se recomienda usarlos con moderación y buscar alternativas cuando se trabaja con grandes volúmenes de datos. En muchos casos, el uso de conjuntos (set) o diccionarios (dict) permite reemplazar ciclos anidados por operaciones mucho más eficientes.

Los ciclos anidados permiten procesar datos organizados en varios niveles. Para controlar los valores que recorren los bucles, Python incorpora funciones de rango y recorrido.

4. Operadores de rango de datos: bucles

Cuando se trabaja con ciclos en Python, frecuentemente es necesario generar secuencias de números o recorrer rangos específicos de valores. Los operadores y funciones de rango de datos cumplen este propósito, proporcionan los valores sobre los cuales se ejecutan los bucles. La función `range()` es la más utilizada en este contexto, aunque Python ofrece otras herramientas que complementan su funcionalidad, como `enumerate()`, `zip()`, `reversed()` y `sorted()`.

Dominar estas funciones permite construir bucles precisos, recorrer colecciones con eficiencia y manipular índices de forma adecuada. Además, facilita la generación de datos de prueba, la implementación de algoritmos matemáticos y el procesamiento ordenado de información. En el desarrollo de software, su uso es frecuente para controlar recorridos y organizar datos.

Las funciones de recorrido facilitan la generación y organización de secuencias. La primera de ellas es `range()`, empleada para producir series de números enteros.

4.1. Función `range()`

La función `range()` genera una secuencia inmutable de números enteros que se utiliza principalmente en ciclos `for`. Acepta uno, dos o tres argumentos, que permiten configurar el inicio, el fin y el incremento de la secuencia generada. Es importante recordar que el valor final especificado no se incluye en la secuencia; esta característica se conoce como intervalo semiabierto.

La versatilidad de `range()` radica en que admite uno, dos o tres argumentos, con efectos distintos en la secuencia generada. La tabla resume estas formas de uso con un ejemplo de cada una.

Tabla 9. Formas de uso de la función `range()`

Sintaxis	Descripción	Ejemplo
<code>range(fin)</code>	Genera números desde 0 hasta <code>fin-1</code> .	<code>range(5) → 0, 1, 2, 3, 4</code>
<code>range(inicio, fin)</code>	Genera números desde inicio hasta <code>fin-1</code> .	<code>range(2, 6) → 2, 3, 4, 5</code>
<code>range(inicio, fin, paso)</code>	Genera números con incremento definido.	<code>range(0, 10, 2) → 0, 2, 4, 6, 8</code>

Dominar las tres formas de `range()` permite generar exactamente la secuencia de números que requiere cada ciclo, recordando siempre que el valor final queda excluido. Esta función es la herramienta más utilizada para controlar las iteraciones de un bucle `for`.

Para comprender la aplicación de la función `range()`, se presentan dos códigos: el primero muestra la generación de secuencias numéricas y el segundo explica su uso eficiente de la memoria:

- **Código. Formas de uso de range()**

Ejemplos con range()

```
>>> list(range(5))
```

```
[0, 1, 2, 3, 4]
```

```
>>> list(range(2, 8))
```

```
[2, 3, 4, 5, 6, 7]
```

```
>>> list(range(0, 20, 5))
```

```
[0, 5, 10, 15]
```

Secuencia decreciente con paso negativo

```
>>> list(range(10, 0, -2))
```

```
[10, 8, 6, 4, 2]
```

Cuenta regresiva

```
>>> list(range(5, 0, -1))
```

```
[5, 4, 3, 2, 1]
```

El código genera secuencias con diferentes valores de inicio, límite e incremento, incluido el recorrido descendente. En todos los casos, el valor final se excluye de la secuencia.

- **Código. Eficiencia de range() en memoria**

range() es eficiente en memoria

```
>>> numeros = range(1, 1000001) # Rango de un millón
```

```
>>> type(numeros)
```

```
<class 'range'>
```

```
>>> # No se almacenan los números, solo la regla para generarlos
```

El código crea un rango de un millón de números sin almacenarlos individualmente. Python conserva únicamente la regla necesaria para generarlos durante el recorrido.

La función range() define el inicio, el límite y el incremento de una secuencia. A continuación, se analiza su integración con el ciclo for para controlar un número determinado de repeticiones.

4.2. Uso de range() en ciclos for

La combinación del ciclo for con la función range() es una de las construcciones más comunes en Python. Permite ejecutar un bloque de instrucciones un número exacto de veces, controlando con precisión el valor de la variable de iteración. Esta combinación es especialmente útil en problemas matemáticos, generación de datos y recorrido indexado de secuencias.

Este recurso permite repetir una acción el número de veces requerido, sin depender de una lista existente. Se emplea en tareas frecuentes de cálculo y

generación de datos de prueba, como sumar series numéricas, construir tablas, generar identificadores o ejecutar procesos con repeticiones definidas.

Para comprender cómo el ciclo for y la función range() controlan repeticiones numéricas, a continuación, se presentan ejemplos para mostrar los primeros números de una secuencia, calcular una suma, construir una tabla de multiplicar, obtener un factorial y generar potencias de 2:

- **Código. Ciclo for con range(): primeros 5 números.**

```
# Imprimir los primeros 5 números

for i in range(5):

    print(i)

# Calcular la suma de los números del 1 al 100

suma = 0

for numero in range(1, 101):

    suma += numero

print(f'La suma es: {suma}')

# Tabla de multiplicar del 5

for i in range(1, 11):

    print(f'5 x {i} = {5 * i}')
```

El primer código muestra los números del 0 al 4. Los dos códigos siguientes calculan la suma de los números del 1 al 100 y generan la tabla de multiplicar del 5, desde 5×1 hasta 5×10 .

- **Código. Programa: calcular factorial con for y range().**

```
# Programa: calcular factorial

n = int(input('Ingrese un número: '))

factorial = 1

for i in range(1, n + 1):

    factorial *= i

print(f'{n}! = {factorial}')

# Ejemplo: 5! = 120
```

El código multiplica los números enteros desde 1 hasta el valor ingresado. Por ejemplo, si se ingresa 5, calcula $1 \times 2 \times 3 \times 4 \times 5$ y obtiene 120.

- **Código. Programa: generar potencias de 2.**

```
# Programa: generar potencias de 2

print('Potencias de 2:')

for i in range(1, 11):

    print(f'2^{i} = {2 ** i}')

# Salida:

# 2^1 = 2
```

```
# 2^2 = 4
```

```
# 2^3 = 8
```

```
# ...
```

```
# 2^10 = 1024
```

El código eleva el número 2 a los exponentes comprendidos entre 1 y 10. En cada repetición, muestra la potencia y el resultado correspondiente.

La combinación de `for` y `range()` controla repeticiones numéricas. Cuando también se necesita conocer la posición de cada elemento de una secuencia, resulta conveniente utilizar `enumerate()`.

4.3. Función `enumerate()`

Cuando se recorre una secuencia con un ciclo `for` pero también se necesita conocer la posición (índice) de cada elemento, se utiliza la función `enumerate()`. Esta función asocia a cada elemento un número consecutivo que comienza en 0, generando pares (índice, valor). Resulta mucho más elegante que manejar un contador manualmente.

Conocer la posición de cada elemento facilita tareas como las siguientes:

- Numerar las opciones de un menú.
- Mostrar resultados según su orden de aparición.
- Identificar la fila en la que se encuentra un dato.

En estos casos, la función proporciona la posición y el valor de cada elemento, sin necesidad de crear ni actualizar un contador adicional.

Para comprender cómo la función `enumerate()` relaciona cada elemento con su posición, a continuación, se presentan tres códigos: numeración de una lista, comparación con `range(len())` y construcción de un menú dinámico:

- **Código. Uso de `enumerate()` para índice y valor.**

```
# Usar enumerate para obtener índice y valor
```

```
frutas = ['manzana', 'pera', 'uva']
```

```
for indice, fruta in enumerate(frutas):
```

```
    print(f'{indice}: {fruta}')
```

```
# Salida:
```

```
# 0: manzana
```

```
# 1: pera
```

```
# 2: uva
```

```
# También se puede iniciar en un número diferente
```

```
for num, fruta in enumerate(frutas, start=1):
```

```
    print(f'{num}. {fruta}')
```

```
# Salida:
```

```
# 1. manzana
```

```
# 2. pera
```

3. uva

El código recorre una lista de frutas y muestra cada elemento con su posición. La numeración puede comenzar en 0 o modificarse para iniciar en 1.

- **Código. Comparación: enumerate() vs. range(len()).**

Forma no recomendada (usando len y range)

```
for i in range(len(frutas)):
```

```
    print(f'{i}: {frutas[i]}')
```

Forma recomendada (usando enumerate)

```
for i, fruta in enumerate(frutas):
```

```
    print(f'{i}: {fruta}')
```

Ambas formas permiten obtener la posición y el valor de cada fruta. Sin embargo, enumerate() expresa esta tarea de manera más directa, clara y fácil de leer.

- **Código. Programa: menú dinámico con enumerate().**

Programa: menú dinámico numerado

```
opciones = ['Registrarse', 'Iniciar sesión',
```

```
            'Recuperar clave', 'Salir']
```

```
print('===== MENÚ =====')
```

```
for numero, opcion in enumerate(opciones, start=1):
```

```
    print(f'{numero}. {opcion}')
```



```
eleccion = int(input('Elija una opción: '))  
  
print(f'Usted eligió: {opciones[eleccion - 1]}')
```

El código enumera las opciones del menú desde 1 y solicita seleccionar una de ellas. Después, utiliza el número ingresado para mostrar la opción correspondiente.

La función `enumerate()` relaciona cada elemento con su posición. Para recorrer de manera simultánea datos almacenados en dos o más secuencias, Python dispone de la función `zip()`.

4.4. Función `zip()`

La función `zip()` combina dos o más secuencias en pares de elementos correspondientes. Se utiliza para recorrer varias secuencias en paralelo dentro de un mismo ciclo. El resultado contiene tantos pares como elementos tenga la secuencia más corta.

Para comprender la utilidad de la función `zip()`, se presentan algunos casos en los que permite relacionar datos almacenados en listas paralelas:

- **Nombres y edades:** relaciona cada persona con su edad.
- **Productos y precios:** asocia cada producto con su valor.
- **Estudiantes y calificaciones:** vincula cada estudiante con su resultado.

De esta manera, `zip()` permite recorrer y procesar datos relacionados de forma conjunta, sin utilizar índices manuales.

Su principal ventaja es la legibilidad, en lugar de acceder a cada lista por separado mediante índices, el ciclo for recibe directamente una variable por cada secuencia combinada. Esta forma de recorrer datos relacionados resulta clara y reduce los errores de desfase entre listas.

Para comprender cómo zip() relaciona y recorre datos almacenados en varias secuencias, a continuación, se presentan ejemplos con dos y tres listas, así como con secuencias de diferente longitud:

- **Código. Uso de zip() para combinar listas.**

```
# Combinar dos listas con zip

nombres = ['Ana', 'Luis', 'María']

edades = [25, 30, 22]

for nombre, edad in zip(nombres, edades):

    print(f'{nombre} tiene {edad} años')

# Salida:

# Ana tiene 25 años

# Luis tiene 30 años

# María tiene 22 años
```

El código relaciona cada nombre con la edad que ocupa la misma posición en la segunda lista. Luego, recorre las parejas y muestra la información de cada persona.

- **Código. Uso de zip() con tres listas.**

```
# Combinar tres listas
```

```
nombres = ['Ana', 'Luis', 'Pedro']
```

```
ciudades = ['Ibagué', 'Bogotá', 'Medellín']
```

```
profesiones = ['Ingeniera', 'Médico', 'Arquitecto']
```

```
for nombre, ciudad, profesion in zip(nombres, ciudades, profesiones):
```

```
    print(f'{nombre} es {profesion} en {ciudad}')
```

El código agrupa el nombre, la ciudad y la profesión ubicados en la misma posición de las tres listas. Después, recorre cada grupo y presenta la información asociada.

- **Código. Comportamiento de zip() con longitudes diferentes.**

```
# zip() con secuencias de diferente longitud
```

```
lista_a = [1, 2, 3, 4, 5]
```

```
lista_b = ['a', 'b', 'c']
```

```
for num, letra in zip(lista_a, lista_b):
```

```
    print(f'{num}-{letra}')
```

```
# Salida:
```

```
# 1-a
```

```
# 2-b
```

3-c

(los elementos 4 y 5 de lista_a son ignorados)

El código relaciona los números con las letras hasta que termina la lista más corta. Por esta razón, muestra tres parejas e ignora los números 4 y 5 de la primera lista.

Una vez comprendido cómo `zip()` relaciona elementos de varias secuencias, es importante reconocer que los datos también pueden requerir un orden específico para facilitar su análisis. Por ello, a continuación, se presentan las funciones `reversed()` y `sorted()`, utilizadas para invertir recorridos y ordenar elementos, respectivamente.

4.5. Funciones `reversed()` y `sorted()`

Las funciones `reversed()` y `sorted()` son útiles cuando se desea recorrer una secuencia en un orden diferente al original. La función `reversed()` invierte el orden de los elementos, mientras que `sorted()` devuelve una lista con los elementos ordenados.

Presentar la información en un orden distinto al de captura es una necesidad constante: mostrar los últimos registros primero, ordenar alfabéticamente una lista de nombres o clasificar calificaciones de mayor a menor. Estas funciones permiten obtener ese orden en el momento de recorrer los datos.

Una característica clave es que ninguna de las dos modifica la secuencia original: ambas devuelven el resultado como un nuevo recorrido u objeto, de modo que los

datos iniciales se conservan. Distinguir entre invertir y ordenar, y entre orden ascendente y descendente, permite elegir la función adecuada para cada caso.

- **Código. Uso de las funciones `reversed()` y `sorted()`.**

```
# Recorrer una lista en orden inverso
```

```
numeros = [1, 2, 3, 4, 5]
```

```
for num in reversed(numeros):
```

```
    print(num)
```

```
# Salida: 5, 4, 3, 2, 1
```

```
# Recorrer una lista ordenada ascendentemente
```

```
desordenados = [5, 2, 8, 1, 9, 3]
```

```
for num in sorted(desordenados):
```

```
    print(num)
```

```
# Salida: 1, 2, 3, 5, 8, 9
```

```
# Orden descendente con sorted()
```

```
for num in sorted(desordenados, reverse=True):
```

```
    print(num)
```

```
# Salida: 9, 8, 5, 3, 2, 1
```

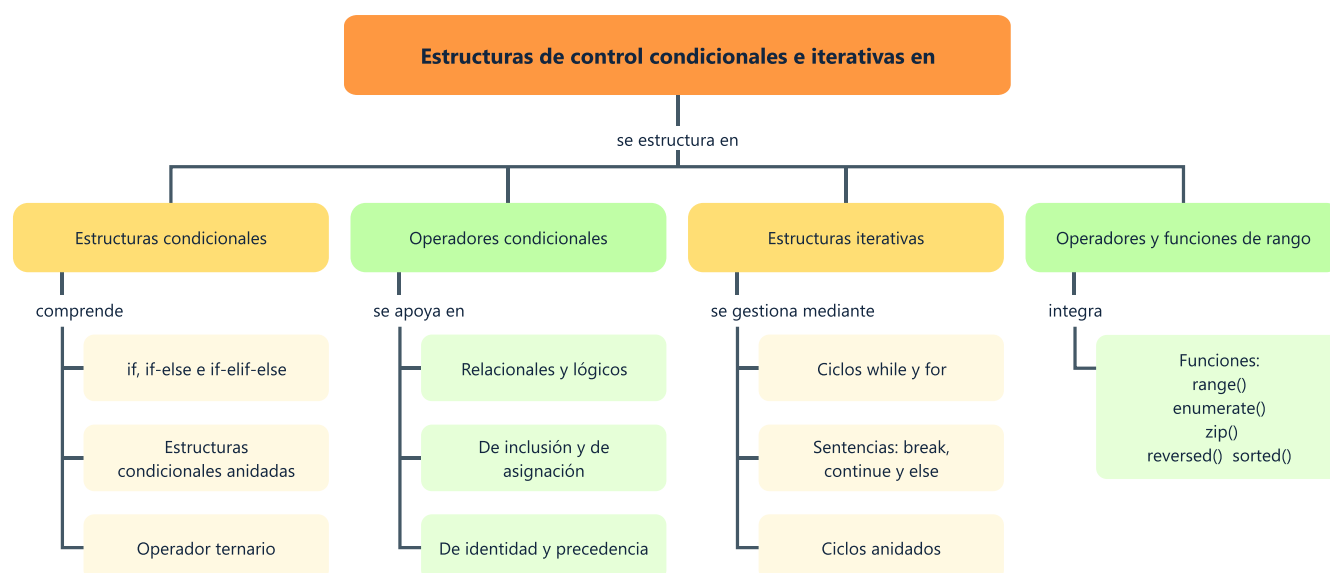
Ambas funciones crean una nueva secuencia sin modificar la original. Esto es importante cuando se necesita preservar el orden original de los datos para otros procesos posteriores.

Síntesis

El componente formativo presenta las estructuras de control condicionales de Python: if, if-else, if-elif-else, las condiciones anidadas y el operador ternario. Estas estructuras permiten orientar la ejecución de un programa mediante expresiones construidas con operadores relacionales, lógicos, de inclusión, de asignación y de identidad. También se aborda la precedencia, que determina el orden de evaluación de los operadores.

Asimismo, se estudian los ciclos while y for, las sentencias break, continue y else, y los ciclos anidados. Este aprendizaje se complementa con las funciones range(), enumerate(), zip(), reversed() y sorted(), utilizadas para controlar repeticiones y recorrer datos. A continuación, el mapa conceptual organiza de manera visual los ejes temáticos del componente formativo y las relaciones que los articulan

A continuación, el mapa conceptual organiza de manera visual los ejes temáticos del componente formativo y las relaciones que los articulan.



Glosario

Break: sentencia de Python que interrumpe un ciclo de forma anticipada, sin importar si la condición sigue siendo verdadera.

Ciclo: estructura iterativa que repite un bloque de instrucciones varias veces mientras se cumple una condición o mientras existan elementos por recorrer.

Condición: expresión booleana que al evaluarse produce un valor True o False y determina el comportamiento de una estructura de control.

Elif: palabra reservada de Python, abreviatura de else if, que permite encadenar varias condiciones dentro de una estructura condicional.

Else: palabra reservada de Python que define el bloque que se ejecuta cuando ninguna de las condiciones previas se cumple.

Enumerate: función integrada de Python que añade un índice consecutivo a cada elemento de una secuencia durante su recorrido.

For: estructura iterativa de Python que recorre los elementos de una secuencia en orden.

Operador lógico: operador que combina expresiones booleanas; en Python son and, or y not.

Range: función integrada de Python que genera una secuencia inmutable de números enteros, útil para controlar repeticiones en un ciclo.

While: estructura iterativa de Python que repite un bloque de instrucciones mientras una condición se mantenga verdadera.

Referencias bibliográficas

Cuevas, A. (2017). Python 3: Curso práctico. Ediciones de la U.

Guzdial, M. y Ericson, B. (2013). Introducción a la computación y programación con Python. Pearson Educación.

Hinojosa, A. (2016). Python paso a paso. Ediciones de la U.

Pérez, A. (2016). Python fácil. Alfaomega Grupo Editor.

Salazar, P. (2019). Empezando a programar en Python. Editorial Escuela Colombiana de Ingeniería.

Lutz, M. (2013). Learning Python (5.ª ed.). O'Reilly Media.

Python Software Foundation. (s.f.). The Python Tutorial.

<https://docs.python.org/3/tutorial/>

Van Rossum, G., Warsaw, B. y Coghlan, N. (2001). PEP 8 – Style Guide for Python Code. <https://peps.python.org/pep-0008/>

Créditos

Nombre	Cargo	Centro de Formación y Regional
Claudia Johanna Gómez Pérez	Profesional G06. Responsable Ecosistema Virtual de Recursos Educativos Digitales	Centro Agroturístico - Regional Santander
Diana Rocío Possos Beltrán	Responsable de línea de producción	Centro de Comercio y Servicios - Regional Tolima
Solanlly Sánchez Melo	Experta temática	Centro de Comercio y Servicios - Regional Tolima
Gloria Lida Alzate Suárez	Evaluadora instruccional	Centro de Comercio y Servicios - Regional Tolima
José Jaime Luis Tang Pinzón	Diseñador de contenidos digitales	Centro de Comercio y Servicios - Regional Tolima
Manuel Felipe Echavarria Orozco	Desarrollador full stack	Centro de Comercio y Servicios - Regional Tolima
Gilberto Junior Rodríguez Rodríguez	Animador y productor audiovisual	Centro de Comercio y Servicios - Regional Tolima
Jorge Eduardo Rueda Peña	Evaluador de contenidos inclusivos y accesibles	Centro de Comercio y Servicios - Regional Tolima
Javier Mauricio Oviedo	Validador y vinculator de recursos educativos digitales	Centro de Comercio y Servicios - Regional Tolima